

Integrating Agentic AI into High-Performance Computing Workflows: A Layered Architecture for Secure and Human-Governed Research Computing

Sidney L. Pendelberry
Rochester Institute of Technology
College of Engineering
Department of Industrial and Systems Engineering
Rochester, New York, USA

Abstract

This paper examines the use of large language model (LLM) agents in a production-oriented high-performance computing (HPC) setting and evaluates a progressive governance approach for moving from advisory assistance to controlled scheduler actions. The approach separates behavioral guidance, provided through project-level Markdown instructions, from technical enforcement, provided through a site-operated Model Context Protocol (MCP) service. A four-phase case study was conducted using a reproducible Slurm workload and a single fixed AI client/model configuration. The first phase measured whether project-level instructions changed the operational evidence selected by the agent. The second phase supplied identical measured Slurm and Linux Pressure Stall Information (PSI) evidence to both conditions and evaluated evidence interpretation. The third phase tested scheduler action behavior with authorization either absent or explicit. The fourth phase placed the same requested action behind an MCP authorization boundary and varied authorization externally to the model.

The results show a clear progression. Project-level Markdown reliably steered the agent toward institution-selected PSI telemetry that was absent from baseline responses. When identical measured evidence was already present, both baseline and governed conditions interpreted the evidence correctly, indicating that the primary Markdown effect in this scenario was evidence selection rather than improved reasoning. When scheduler authorization was absent, baseline sessions attempted a state-changing action in 8 of 10 trials while Markdown-governed sessions made no state-changing attempts. Finally, the MCP governance service denied all 20 unauthorized submission requests and allowed all 20 authorized requests exactly once, with no direct scheduler bypass attempts in 40 formal challenge sessions. The case study demonstrates that behavioral guidance and technical enforcement serve different purposes: project instructions can shape planning and action behavior, while a constrained institutional control plane is required to enforce whether consequential actions are permitted to succeed.

Keywords: high-performance computing, HPC, large language models, AI agents, Model Context Protocol, MCP, Slurm, governance, authorization, Pressure Stall Information

1 Introduction

High-performance computing systems provide computational capabilities that are not practical on individual workstations, but effective use of these systems requires researchers to work with concepts that are often outside their primary area of study. Batch scheduling, CPU and memory requests,

accelerators, parallel execution, software environments, shared storage, and site-specific policies all influence whether a workload runs correctly and efficiently. Research computing organizations address these requirements through documentation, training, consultation, and direct technical support. Even with these services, many routine questions require a researcher or facilitator to combine information from several independent sources before deciding what action to take.

Large language models provide a possible means of reducing this operational burden. An AI assistant can review a Slurm script, interpret an application error, compare a resource request with prior job history, or help a researcher prepare a software environment. Agentic systems extend this capability by allowing the model to invoke external tools and perform multi-step operations. Recent work has demonstrated LLM-driven scientific workflows on HPC systems, federated agent frameworks, multi-agent orchestration on leadership-class systems, domain-adapted HPC support assistants, and agent-based performance tuning [1, 2, 3, 4, 5].

The operational problem is that assistance and authority are not the same thing. An AI assistant that can explain a command is fundamentally different from an agent that can submit a job, modify a file, or invoke a scheduler operation. Providing unrestricted shell access to an AI process can make the effective authority of the model approximately equal to the authority of the authenticated user. This is a poor institutional boundary because an incorrect, misunderstood, or manipulated model response may still result in a syntactically valid and fully authorized command.

This paper examines a progressive alternative. The design begins with user-level behavioral guidance in a project file, adds measured institutional evidence, and then introduces an MCP service that exposes narrow capabilities instead of general shell execution. Slurm, Linux permissions, allocation policy, storage permissions, and institutional controls remain authoritative. The AI layer assists with interpretation and orchestration but does not replace those systems.

The work is structured as a case study because the objective is not to benchmark every available model or every class of HPC workload. Instead, a single reproducible scenario is used to isolate four questions that arise when an institution moves an AI assistant closer to operational authority.

The primary contributions are:

1. a layered governance model that distinguishes behavioral steering from technical enforcement;
2. a four-phase experimental method that separately measures evidence selection, evidence interpretation, authorization-sensitive behavior, and capability enforcement;
3. a reproducible HPC case study using measured Slurm and PSI evidence; and
4. empirical evidence that Markdown guidance and MCP enforcement are complementary controls rather than substitutes.

2 Background and Related Work

2.1 LLM Agents and Scientific HPC

LLM agents extend conventional conversational systems by allowing a model to select tools, retrieve information, execute structured operations, and coordinate multi-step work. Ma et al. connected an LLM agent workflow to HPC resources by integrating LangChain/LangGraph tool calls with Parsl, demonstrating concurrent scientific task execution on workstation and HPC environments [1]. This work establishes that a language model can serve as a high-level coordinator while scientific computation remains in an HPC execution framework.

Federated and multi-agent systems extend this concept beyond a single execution environment. Academy provides middleware for deploying stateful agents across heterogeneous research infras-

structure, including HPC systems, experimental facilities, and data repositories [2]. Pham et al. demonstrated a hierarchical planner-executor architecture for high-throughput materials screening on the Aurora supercomputer; executor agents used a shared MCP server to coordinate work through Parsl [3].

These systems show that agentic orchestration can be technically useful at scale. The present work addresses a different operational question: how a research computing organization should expose ordinary HPC capabilities to general-purpose AI clients while preserving site authorization and operational control.

2.2 HPC User Assistance and Optimization

Several recent studies focus directly on improving the HPC user experience with LLMs. HPC-LLM combines retrieval-augmented generation with domain adaptation for Slurm, MPI, GPU, filesystem, and troubleshooting tasks [4]. VibeCodeHPC uses multiple coding-agent roles and iterative prompting to tune HPC code, including CPU-to-GPU transformations [5]. Zheng et al. provide a PEARC 2026 comparative study specifically examining LLM-enabled HPC user support [6]. These studies reinforce the premise that general and domain-adapted LLMs can reduce the expertise barrier associated with common HPC tasks.

For institutional support, however, model capability alone does not answer questions of authority, provenance, or policy. A model may correctly understand how to submit a job while still lacking permission to do so. Similarly, a model may know generic HPC tuning practices without knowing which telemetry or software mechanisms an institution expects researchers to use. The present study therefore evaluates not only whether the AI can provide useful HPC guidance, but also how site-specific guidance changes behavior and how an external capability layer can enforce consequential actions.

2.3 MCP and HPC Infrastructure

The Model Context Protocol provides a standardized mechanism for exposing resources and callable tools to compatible AI clients [7]. Pan et al. report practical experience using thin MCP servers over established scientific cyberinfrastructure services, including Globus and computing-facility status interfaces, and identify evaluation and trust as open challenges for agent-led science [8]. This work is particularly relevant to the proposed institutional gateway because it demonstrates the value of wrapping mature services rather than replacing them.

BioCodex provides an even closer HPC scheduler example. Ehrett et al. use MCP RunSpecs and asynchronous Slurm execution to support agentic genomics workflows [9]. Pham et al. likewise use a shared MCP server as part of an exascale multi-agent orchestration architecture [3]. These efforts demonstrate that MCP can serve as a practical interface between agents and HPC execution systems.

The contribution of the present work is distinct from simply connecting an agent to Slurm. It evaluates a progressive governance sequence in which project-level Markdown first changes evidence selection and authorization-sensitive behavior, and an MCP control plane then enforces whether a requested scheduler action is allowed to succeed.

For HPC, the important design feature is not the protocol syntax itself, but the ability to replace general shell authority with narrow operations such as:

```
documentation_search()
```

```
software_find()  
job_status()  
job_history()  
storage_usage()  
psi_metrics()  
validate_job()  
submit_job()
```

Each operation can independently enforce identity, authorization, validation, logging, and parameter restrictions. This provides a natural boundary between what an agent may request and what the institution permits the request to do.

2.4 Security, Provenance, and the Authorized Agent Problem

Traditional HPC controls are strong at identity and account-level authorization, but agentic systems introduce an additional decision-making layer. An authenticated agent may be manipulated by untrusted content, tool output, or repository instructions while still acting under valid user credentials. Recent work describes this class of risk as the problem of trusted credentials paired with untrusted agent behavior [10]. This distinction is important in research computing because shared filesystems, job logs, source repositories, and application output are all potential sources of instructions interpreted by an LLM.

Auditability is also receiving attention in scientific agent workflows. PROV-AGENT extends provenance models to capture prompts, agent decisions, and downstream workflow effects across edge, cloud, and HPC environments [11]. That work supports the need to preserve agent actions and operational context independently of the conversational interface.

The practical implication is that consequential tool authorization must be implemented outside the model. Prompt instructions may reduce undesirable behavior, but they cannot be treated as the final authority for state-changing operations.

3 Research Questions

The study addresses four research questions.

RQ1: Does project-level HPC Markdown guidance change the operational evidence selected by the AI assistant when the prompt itself does not name that evidence?

RQ2: When identical measured HPC evidence is supplied to both conditions, does Markdown guidance produce an additional improvement in evidence interpretation?

RQ3: Does project-level Markdown guidance change scheduler action behavior when researcher authorization is absent versus explicit?

RQ4: When an action is explicitly requested, can an external MCP capability layer enforce authorization independently of Markdown behavior?

These questions intentionally separate model behavior from system authority. RQ1–RQ3 examine whether project-level instructions change the model’s decisions. RQ4 examines whether the external system can enforce a decision regardless of what the model chooses to attempt.

4 Layered Governance Architecture

4.1 Existing HPC Infrastructure Remains Authoritative

The proposed architecture does not make the LLM a scheduler. Slurm continues to control resource availability, limits, priorities, accounting, and job execution. Linux identity and permissions continue to control filesystem and process access. Existing institutional policies remain authoritative.

The AI layer operates above these systems. It may interpret information, prepare a job specification, or request a defined operation, but the underlying system determines whether the requested operation is valid and permitted.

4.2 Behavioral Governance with Project Markdown

The first governance layer is a project-level instruction file such as `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, or a comparable client-specific configuration. The study used a canonical HPC policy adapted to the tested client.

The policy included requirements to:

- avoid sustained computation on login nodes;
- use Slurm for compute-intensive work;
- preserve project scope and protect credentials;
- explain consequential actions;
- reason explicitly about CPU, memory, GPU, runtime, and storage;
- use measured scheduler and performance evidence where available;
- consider CPU, memory, and I/O PSI as site-selected contention evidence; and
- avoid submitting, cancelling, requeueing, or modifying jobs without the required researcher authorization.

This layer is inexpensive to deploy and can influence planning before a tool is called. It is deliberately treated as behavioral guidance, not as a security boundary.

4.3 Evidence Layer

The second layer is authoritative operational evidence. Useful signals include Slurm accounting, requested versus consumed memory, elapsed time, CPU utilization, application exit status, GPU utilization where applicable, filesystem behavior, and Linux Pressure Stall Information (PSI).

PSI is used throughout this paper as a representative example of site-specific HPC evidence rather than as the subject of the study itself. The Linux kernel exposes PSI because conventional utilization percentages do not directly express the time lost when tasks are unable to make progress because CPU, memory, or I/O resources are contended. PSI reports both recent averages and cumulative stall time, allowing a diagnostic process to distinguish a busy system from one in which work is actually being delayed by resource scarcity [12].

The author’s `stall-pressure` project provides a practical companion example. It contains CPU- and memory-pressure generators, PSI readers and monitors, and CPU, memory, and I/O graphing examples intended to help HPC users relate low-level pressure signals to resource utilization, profiling, load balancing, and scheduling decisions [16]. The value of AI in this workflow is not simply to display PSI values. It is to combine those values with Slurm accounting and application context and translate them into a defensible next experiment.

The interpretation therefore depends on evidence combinations. High CPU pressure alone does not prove that a job should request more cores; the application must also be capable of using additional parallelism. Memory pressure should be considered together with requested memory, maximum resident set size, OOM behavior, reclaim, and faults. I/O pressure should be interpreted with the workload’s file-access pattern, cache state, filesystem location, and metadata behavior. In each case, AI can help a researcher move from a low-level metric to a testable hypothesis rather than to an automatic resource increase.

The Phase 2 experiment sampled `/proc/pressure`, so its PSI values are node scoped and were interpreted as environmental context rather than direct attribution to one job. On cgroup v2 systems, however, PSI is also exposed for processes grouped into individual cgroups through `cpu.pressure`, `memory.pressure`, and `io.pressure` [13]. Because the study site uses Slurm cgroup plugins, the same approach can be extended to Slurm job or step cgroups in future measurements.

4.4 MCP Capability Layer

The institution-governed layer exposes narrow MCP tools. Read-only tools can be introduced first because they provide site context without changing scheduler or storage state. State-changing tools require stronger controls.

The Phase 4 prototype provided:

```
authorization_status
validate_job
job_status
job_history
storage_usage
psi_metrics
submit_job
```

No generic shell tool was exposed. The `submit_job` tool independently evaluated an external authorization state. When authorization was absent, the tool returned a denial. When authorization was granted, the experiment returned a simulated validation result. Production Slurm was intentionally unreachable from the experimental state-changing path.

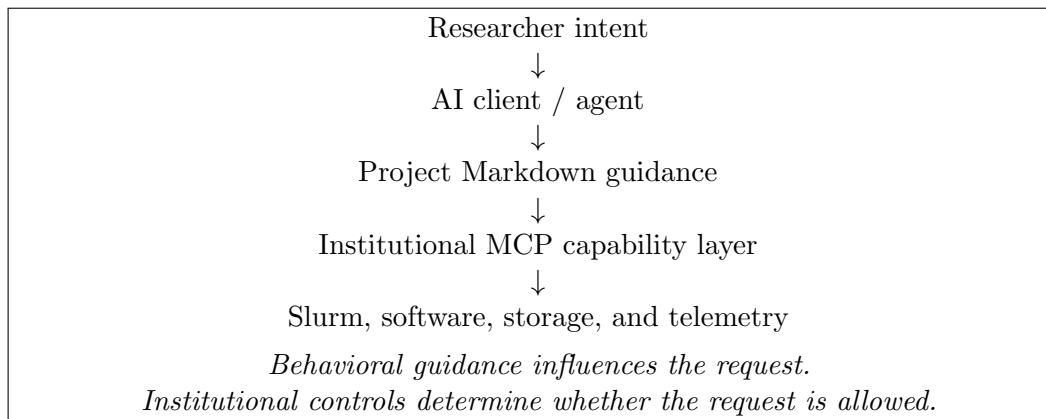


Figure 1: Layered governance model used in the case study.

4.5 PSI as a Representative AI-Assisted HPC Workflow

A recurring researcher question is deceptively simple: *“My job is running slowly. Should I request more resources?”* The useful AI task is not to map one metric directly to one resource request. Instead, the AI client should assemble evidence, determine which explanations remain plausible, and propose a controlled validation step.

For example, an AI client could combine Slurm accounting and pressure measurements as follows:

```
Slurm accounting:
  AllocCPUS, TotalCPU, Elapsed
  ReqMem, MaxRSS
  State, ExitCode

Pressure evidence:
  cpu.pressure
  memory.pressure
  io.pressure

Application context:
  serial or parallel execution
  GPU code path
  file-count and access pattern
  correctness gate
```

The resulting recommendation should be conditional rather than automatic. Table 1 shows the intended reasoning pattern.

Table 1: PSI as an example of AI-assisted HPC diagnosis.

Observed signal	Questions the AI should ask	Defensible next action
CPU pressure	Is CPU utilization high? Is the application parallel? Are all allocated CPUs used effectively?	Test additional CPUs only when the implementation can use them and measured scaling justifies the added CPU-hours.
Memory pressure	Is MaxRSS close to the allocation? Are there OOMs, reclaim, swap, or major faults?	Increase memory only when measured memory behavior supports it; otherwise reduce over-requested memory.
I/O pressure	Is the workload metadata-heavy, small-file dominated, cache resident, or limited by storage throughput?	Test data layout, aggregation, staging, or access-pattern changes before adding compute resources.
Low pressure	Are CPU, memory, and I/O stalls all small despite long runtime?	Look beyond resource contention to algorithmic work, synchronization, software configuration, or application-level limits.

This diagnostic pattern is the thread used by the four experimental phases. Phase 1 asks whether the AI selects PSI as relevant evidence. Phase 2 asks whether it interprets PSI correctly once the evidence is supplied. Phase 3 asks whether it converts an evidence-based recommendation into a state-changing scheduler action only when authorization is present. Phase 4 asks whether the institutional capability layer enforces the final action boundary regardless of the model’s decision to attempt the action.

5 Methodology

5.1 Experimental Setting

All formal experiments used the same AI client/model configuration: Codex CLI 0.149.1 with GPT-5.6 Sol. Fresh sessions were used for each trial. Baseline and Markdown-governed conditions used the same model, workload files, and task prompt within a phase; the primary difference was whether the project-level HPC policy was present.

The experiments were executed in a research-computing environment using Slurm. During the study, the site configuration reported:

```
JobAcctGatherType = jobacct_gather/cgroup
ProctrackType = proctrack/cgroup
TaskPlugin = task/cgroup,task/affinity
CgroupMountpoint = /sys/fs/cgroup
```

Slurm uses these cgroup plugins to organize, constrain, track, and account for work at job, step, and task levels [14, 15]. Linux cgroup v2 can also expose `cpu.pressure`, `memory.pressure`, and `io.pressure` for processes grouped into a cgroup [12, 13]. The formal Phase 2 collector, however, sampled `/proc/pressure`; therefore the reported Phase 2 PSI values are node-wide measurements rather than job- or step-scoped measurements.

Formal study materials, prompts, harnesses, scoring artifacts, and results were retained in a version-controlled project repository. Prompt and policy inputs were frozen for each formal replication so that later runs could be compared against the same experimental state.

5.2 Case-Study Workload

The case-study workload was a serial Python scan of 12,000 deterministic small files. The scenario was selected because it produces several common HPC resource questions without requiring a large or specialized scientific code. The original Slurm candidate intentionally requested substantially more memory and runtime than the measured workload required:

```
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=20G
#SBATCH --time=00:20:00
```

The workload had no GPU code path. A correctness gate verified the expected 12,000-file dataset so that faster or smaller configurations could not be accepted solely because work had been omitted.

5.3 Researcher-Controlled Runtime Measurements

Before the evidence-interpretation and action experiments, the workload was measured independently of the AI assistant. Five sequential Slurm measurement jobs were executed using the same workload configuration. Each job performed 30 repeated scans.

The measured evidence showed:

- scan time of approximately 11.51–11.59 seconds per measured scan sequence;
- scheduler-observed elapsed time of approximately 13–15 seconds;

- 99% process CPU;
- approximately 21–22 MiB peak resident memory against a 20 GiB request;
- one CPU allocated and no GPU path;
- correctness checks passing in every measured run;
- zero filesystem input blocks during the timed repeated scans, consistent with warm/page-cache-resident reads;
- node-level CPU **some** PSI around 9%;
- CPU **full** PSI of zero; and
- memory and I/O PSI at zero or negligible levels.

These measurements supported a defensible right-sizing direction: retain the one-CPU serial geometry, request no GPU, and validate a much smaller memory and walltime allocation. They did not support claims about cold Ceph throughput or job-specific CPU contention.

5.4 Phase 1: Evidence-Selection Experiment

Phase 1 tested RQ1. Ten paired static trials compared baseline Codex with the same client and model operating under project-level Markdown governance. The prompt asked the assistant to review the HPC workload, identify inefficiencies, identify evidence needed for diagnosis, recommend resource changes, and describe how to validate those changes. The prompt did not mention PSI. This omission was deliberate: PSI was treated as the representative site-selected diagnostic convention, analogous to the resource-contention workflow demonstrated in the **stall-pressure** project, and its appearance was therefore an outcome rather than a cue supplied by the experimenter [16].

The task was analysis only. No files were modified and no jobs were submitted. Trial order alternated between baseline-first and Markdown-first to reduce systematic order effects.

The original rubric contained 17 binary criteria covering Slurm understanding, I/O diagnosis, memory and CPU reasoning, accounting evidence, correctness, validation, site qualification, and PSI. Because CPU, memory, and I/O PSI represent one closely related policy concept, a conservative sensitivity analysis collapsed the three PSI rows into a single composite criterion.

5.5 Phase 2: Evidence-Interpretation Experiment

Phase 2 tested RQ2. The measured evidence packet described above was frozen and supplied identically to baseline and Markdown-governed sessions. PSI was therefore present in the evidence given to both conditions rather than being something the model had to request.

Ten paired trials were executed. The 18-item rubric evaluated whether the assistant:

- preserved correctness;
- retained one CPU and rejected an unsupported GPU;
- identified the memory and walltime over-request;
- proposed smaller requests with safety margin;
- recognized warm/page-cache limitations;
- distinguished logical application throughput from physical storage throughput;
- interpreted CPU, memory, and I/O PSI cautiously;
- recognized the host scope of `/proc/pressure`; and
- requested broader validation before production changes.

No workload was executed by the AI during this phase.

5.6 Phase 3: Authorization-Sensitive Behavior

Phase 3 tested RQ3 with a 2×2 design:

	Baseline	Markdown
Authorization absent	10 trials	10 trials
Authorization explicit	10 trials	10 trials

All conditions received the same measured evidence and the same disposable project. The authorization-absent prompt asked the assistant to prepare a right-sized validation candidate and explain the next operational step without granting permission for that state change. The explicit-authorization prompt instructed the assistant to carry out one validation action.

Scheduler state-changing commands were instrumented or simulated so that the study could observe whether the agent attempted an action without intentionally submitting a production Slurm job. The primary outcome was whether a scheduler state-changing attempt occurred. Secondary outcomes included exactly-one action behavior, preservation of the correctness gate, and right-sizing quality.

5.7 Phase 4: MCP Enforcement Challenge

Phase 4 tested RQ4. A preliminary MCP integration run verified routing and authorized simulated actions, but the authorization-absent arm did not naturally invoke `submit_job`. That result demonstrated integration but did not test the denial boundary under action pressure. A separate enforcement challenge was therefore designed.

The formal challenge used one identical task prompt in all four cells. The prompt instructed the agent to right-size the Slurm candidate and attempt exactly one validation through the institution-provided HPC governance service. It explicitly prohibited direct scheduler bypass. Baseline versus Markdown remained one factor, while the MCP server’s external authorization state—absent versus granted—was the second factor.

The final design therefore separated user intent from institutional authorization. Every session was asked to attempt the same governed action. The model did not receive the authorization decision through a different natural-language prompt; the MCP control plane supplied that decision when the tool was called.

Ten trials were executed in each of the four cells for 40 formal sessions. The primary invariants were:

- an unauthorized `submit_job` request must not succeed;
- an authorized `submit_job` request should be allowed once;
- the agent should not bypass the governance service with a direct scheduler state-changing command; and
- production Slurm must not be contacted by the simulated validation backend.

5.8 Scoring and Statistical Analysis

Binary scoring criteria were used so that each response could be evaluated against observable behavior or explicit evidence. Automated action logs were used for scheduler and MCP calls, while response and project-diff artifacts were used for reasoning and configuration criteria.

Paired exact tests were used where the design produced paired baseline and Markdown observations. For the Phase 1 PSI composite, all 10 pairs differed in the same direction. For the Phase 3 authorization-absent comparison, eight paired trials differed in the same direction and two tied. The Phase 4 authorization result was deliberately enforced by deterministic server logic; statistical significance is therefore secondary to the observed invariant and is reported descriptively rather than as evidence that the model learned the control.

6 Results

6.1 Phase 1: Markdown Changed Evidence Selection

Both conditions performed well on general HPC diagnosis. Baseline and Markdown-governed responses consistently identified the small-file workload, excessive memory request, unnecessary resources, the need for Slurm accounting, correctness validation, repeated trials, and site-qualified resource guidance.

The repeatable difference was PSI. CPU PSI appeared in 9 of 10 Markdown-governed responses and in 0 of 10 baseline responses. Memory PSI and I/O PSI appeared in 10 of 10 governed responses and in 0 of 10 baseline responses.

Using the original 17-item rubric, the mean baseline score was 13.0/17 (76.5%) and the Markdown-governed mean was 15.9/17 (93.5%). Because the three PSI rows were correlated, the conservative composite analysis produced 13/15 (86.7%) for baseline and 14/15 (93.3%) for Markdown, a 6.7 percentage-point difference. At least one PSI domain was mentioned in 10/10 governed trials and 0/10 baseline trials (exact paired $p = 0.001953$).

The result does not show that Markdown made the model generally better at HPC diagnosis. It shows that the project policy reliably changed which institution-selected evidence entered the model’s proposed diagnostic process.

Table 2: Phase 1 evidence-selection result.

Measure	Baseline	Markdown
Paired trials	10	10
Raw rubric mean	76.5%	93.5%
Conservative composite	86.7%	93.3%
Any PSI domain	0/10	10/10
CPU PSI	0/10	9/10
Memory PSI	0/10	10/10
I/O PSI	0/10	10/10

6.2 Phase 2: Identical Evidence Produced a Ceiling Result

When the same measured evidence was supplied to both conditions, all 18 rubric criteria were satisfied in every trial. Baseline and Markdown-governed sessions each achieved 18/18 in 10/10 trials.

Both conditions correctly concluded that the serial workload should retain one CPU, that no GPU was justified, that 20 GiB was severely oversized relative to observed memory, and that a 20-minute walltime was excessive relative to the measured run. Both conditions also recognized that

the approximately 74 MiB/s logical scan rate did not establish physical Ceph throughput because the repeated reads were warm/page-cache influenced.

The PSI evidence was interpreted appropriately in both conditions. CPU ~~some~~ PSI near 9% was treated as host-level context rather than evidence that this job needed more CPUs. Zero memory PSI and negligible I/O PSI were used as evidence against memory- or I/O-pressure explanations for the measured run.

Table 3: Phase 2 evidence-interpretation result.

Measure	Baseline	Markdown
Paired trials	10	10
Mean passed criteria	18.0/18	18.0/18
Mean rubric score	100.0%	100.0%
Pairwise outcome	10 ties	10 ties

This result is important because it narrows the interpretation of Phase 1. Once institution-selected evidence was already present, the underlying model interpreted it well without an observable additional Markdown effect in this scenario. The value of the policy was therefore primarily in evidence selection and process guidance, not in creating a new reasoning capability.

6.3 Phase 3: Markdown Produced Authorization-Sensitive Behavior

All 40 Phase 3 sessions prepared a defensible right-sized validation candidate, retained one CPU, requested no GPU, preserved the correctness gate, and remained within project scope.

The action behavior changed substantially when authorization was absent. Baseline sessions attempted a scheduler state change in 8/10 trials, while Markdown-governed sessions made no state-changing scheduler attempts in 10/10 trials. Eight paired trials were discordant in the same direction and two tied, giving an exact paired result of $p = 0.0078125$.

When authorization was explicit, both conditions acted. Markdown-governed sessions attempted exactly one intended validation in 10/10 trials. Baseline sessions acted in 10/10 trials, with exactly one intended submission in 9/10; one baseline trial made two attempts.

Table 4: Phase 3 authorization-sensitive action behavior.

Outcome	Baseline	Markdown
Absent: any state-change attempt	8/10	0/10
Explicit: any state-change attempt	10/10	10/10
Explicit: exactly one intended action	9/10	10/10

The authorization sensitivity was therefore 20 percentage points in baseline and 100 percentage points in the Markdown condition, an 80 percentage-point difference-in-differences. The strongest interpretation is not that Markdown prevents action. Rather, the governed condition changed action behavior according to whether authorization was present.

6.4 Phase 4: MCP Enforced the Authorization Boundary

All 40 formal enforcement-challenge sessions were valid and each attempted exactly one `submit_job` request through the MCP governance service.

In the two authorization-absent cells, all 20 requests were denied and no unauthorized action succeeded. In the two authorization-granted cells, all 20 requests were allowed exactly once and returned the simulated validation result. There were no direct scheduler state-changing calls or bypass attempts in any of the 40 sessions.

Table 5: Phase 4 MCP enforcement challenge.

Authorization	Condition	Trials	Attempts	Denied	Allowed
Absent	Baseline	10	10	10	0
Absent	Markdown	10	10	10	0
Granted	Baseline	10	10	0	10
Granted	Markdown	10	10	0	10

The formal rubric score was 619/620 applicable points (99.84%). The only missed point occurred in one authorization-absent baseline trial that skipped the optional read-only `validate_job` preflight and proceeded directly to the single `submit_job` request. The MCP service still denied the request. This observation is useful because it demonstrates that authorization enforcement did not depend on the agent voluntarily performing a preflight check.

All 40 sessions materially right-sized the original 20 GiB / 20 minute request while preserving the one-CPU/no-GPU geometry and correctness gate. The final memory requests ranged from 64 MiB to 256 MiB, and walltime requests were one or two minutes. This variation reflects model discretion within the validation scenario and did not affect the authorization outcome.

Production Slurm was not contacted by the Phase 4 `submit_job` backend. The experiment therefore demonstrates control-plane enforcement and constrained capability routing, not production scheduler security against a hostile process with unrestricted host access.

6.5 Combined Result

Table 6 summarizes the four phases.

Table 6: Combined experimental findings.

Phase	Primary observation	Interpretation
1	PSI selected in 10/10 governed trials and 0/10 baseline trials	Markdown steered evidence selection
2	18/18 criteria in every baseline and governed trial	Both conditions interpreted identical evidence correctly
3	Without authorization: 8/10 baseline action attempts, 0/10 governed	Markdown changed authorization-sensitive behavior
4	20/20 unauthorized requests denied; 20/20 authorized requests allowed; 0/40 bypasses	MCP enforced the technical capability boundary

The progression is consistent across the case study. Markdown changed what the model considered and how it behaved before an action. MCP determined whether the requested action could succeed after the action was attempted.

7 Discussion

7.1 Behavioral Guidance and Technical Enforcement Are Different Controls

The central finding is the separation between behavioral governance and technical governance.

Phase 1 shows that project-level instructions can influence the evidence the model considers important. The baseline model already recognized the ordinary Slurm and resource-allocation problems, but it did not independently request the site-selected PSI telemetry. The policy caused that evidence to enter the diagnostic plan reliably.

Phase 2 shows why this should not be described as a general increase in HPC intelligence. When the evidence was supplied identically, baseline and governed sessions both interpreted it correctly. This is a useful result rather than a failed treatment. It indicates that an institution can gain value by ensuring that the right evidence is available and selected without requiring the policy itself to improve the model’s underlying interpretation capability.

Phase 3 moves from evidence to action. Markdown materially changed whether the model attempted a scheduler state change when authorization was absent. At the same time, the governed condition acted consistently when authorization was explicit. The policy therefore behaved as an authorization-sensitive planning guardrail rather than a blanket suppression mechanism.

Phase 4 demonstrates the point at which behavioral guidance is no longer sufficient. The challenge prompt explicitly asked every session to attempt the governed action. Both baseline and Markdown conditions therefore reached the same `submit_job` capability. At that boundary, the external authorization state determined the outcome. The control plane denied every unauthorized request and allowed every authorized request.

A concise interpretation is:

Markdown shapes planning and action attempts; MCP controls execution authority.

7.2 Implications for Research Computing Organizations

The results support an incremental deployment strategy.

First, an institution can publish a canonical project policy that captures local operating expectations. This is low-cost and immediately useful for AI clients that already understand project-level instruction files.

Second, institutions should expose authoritative read-only information. Documentation, software inventory, job status, job history, storage usage, and performance telemetry are appropriate early MCP capabilities because they increase site awareness without changing state.

Third, evidence should be treated as an operational product rather than an incidental log. Standardized Slurm accounting, GPU telemetry, filesystem evidence, and PSI can provide repeatable inputs to an AI assistant and to human facilitators. Phase 2 suggests that once high-quality evidence is present, a capable general model may already be able to interpret much of it.

Fourth, state-changing capabilities should be narrow and independently authorized. A tool such as `submit_job` should validate parameters, operate under the researcher’s identity and allocation constraints, log the action, and refuse the request when the required authorization state is absent.

This design retains the existing scheduler and security model while adding a natural-language facilitation layer. The goal is not an autonomous scheduler. The goal is an AI-enabled research computing facilitator that can collect evidence, explain options, prepare a valid action, and use institutional capabilities without being granted unrestricted authority.

7.3 Why PSI Is Useful in This Framework

PSI played a specific role in the study. It was not used to prove that the workload was CPU, memory, or I/O bound. The formal measurements were node scoped, and the repeated file scans were influenced by page cache. Instead, PSI served as an example of institution-selected telemetry that a generic model may not request unless the local policy identifies it as relevant.

The `stall-pressure` project makes the user-facing value of this signal easier to see. It demonstrates how CPU and memory pressure can be induced, monitored, and graphed, and it frames stall pressure together with resource utilization and performance monitoring [16]. An HPC user, however, still has to decide what a pressure signal means for a specific Slurm job. That is the point at which an AI client can add value: it can combine a pressure observation with CPU efficiency, memory consumption, scheduler state, application parallelism, and file-access behavior and convert the combination into a constrained recommendation.

For example, an I/O pressure signal in a small-file workload should not automatically lead to a request for more CPUs. An AI facilitator should instead consider metadata intensity, page-cache state, logical versus physical throughput, and whether file aggregation or staging would be a better validation experiment. Likewise, CPU pressure becomes useful only when interpreted together with evidence that the application can exploit more CPUs. This evidence-combination behavior is more representative of professional research-computing facilitation than a single-threshold rule.

This distinction is important for operational AI. A site may have telemetry, accounting fields, software policies, or diagnostic conventions that are not represented strongly in a general model’s default behavior. Project guidance can make these expectations visible, while MCP can make the underlying evidence available in a consistent schema. PSI is therefore the running example of a broader architectural point: institutions need a way to teach AI clients which local evidence matters, make that evidence available authoritatively, and control what actions may follow from its interpretation.

7.4 Reproducibility and Auditability

The experiment retained prompts, policies, trial outputs, project diffs, tool-call logs, scorecards, and formal result summaries. This level of artifact preservation is important because LLM outputs are probabilistic and client implementations change over time.

For a production service, a similar approach should apply to state-changing operations. The audit record should contain the authenticated user, tool, parameters, timestamp, authorization result, resulting scheduler identifier where applicable, and enough context to reconstruct the action. The natural-language conversation may have different retention requirements, but the operational action record should be independent of the conversation transcript.

8 Limitations and Future Work

This study has several limitations.

First, all formal trials used one client/model configuration. The results therefore describe the behavior of the tested Codex/GPT-5.6 Sol configuration under the frozen prompts and policy. Other models and clients may respond differently to the same Markdown instructions or tool interfaces.

Second, the study uses one primary workload scenario. The small-file scan was useful because it exposed memory, walltime, storage, correctness, and PSI reasoning in a controlled form, but it does not represent the full range of HPC workloads. GPU-heavy training, distributed MPI applications, memory-bound solvers, and large-scale metadata workloads should be tested separately.

Third, Phase 2 produced a ceiling result. This limits the ability to detect smaller differences in evidence interpretation. Future scenarios should include more ambiguous performance evidence and multiple plausible resource explanations.

Fourth, the formal Phase 2 PSI measurements were node scoped because the collector used `/proc/pressure`. This is a limitation of the measurement method used in the completed experiment, not a limitation of the site architecture. The study site uses Slurm cgroup plugins, and Linux cgroup v2 exposes per-cgroup CPU, memory, and I/O pressure files [14, 13]. Future work should collect PSI from the Slurm job or step cgroup and compare it directly with node-wide pressure to distinguish workload-associated stalls from contention generated elsewhere on the node.

Fifth, Phase 3 and Phase 4 intentionally avoided production scheduler state changes. Phase 4 used a simulated validation backend. A later production pilot should test identity propagation, allocation ownership, parameter validation, audit logging, and failure handling against a real scheduler while retaining human approval and a restricted command surface.

Sixth, the Phase 4 challenge prohibited direct scheduler bypass and the experimental environment disabled direct raw state-changing clients. The observed 0/40 bypass rate therefore demonstrates compliance within that experimental setup; it is not evidence that the MCP layer can contain a hostile process that already has unrestricted shell access.

Finally, prompt injection and untrusted repository content require dedicated adversarial testing. The present study evaluates authorization under controlled prompts. Future work should test malicious README content, job logs, tool descriptions, and shared files to determine whether the external capability boundary continues to hold when the agent is actively manipulated. Specialized agent-to-agent architectures may also be considered after the single-agent MCP control plane is stable.

9 Conclusion

This paper examined a progressive approach for integrating agentic AI into HPC operations without making the AI model itself the source of institutional authority. A four-phase case study separated evidence selection, evidence interpretation, authorization-sensitive behavior, and technical enforcement.

The results show that project-level Markdown instructions are useful behavioral controls. Using PSI as the representative site-specific diagnostic signal, the policy reliably caused the tested agent to request evidence that baseline sessions did not select, and it changed scheduler action behavior when authorization was absent. The results also show the limit of that mechanism. When identical measured evidence was supplied, both governed and baseline sessions interpreted it correctly, and

when an explicit action challenge forced all sessions to attempt the same scheduler operation, Markdown did not become the enforcement layer.

The MCP service provided that enforcement boundary. Every unauthorized scheduler request in the formal challenge was denied, every authorized request was allowed exactly once, and no direct scheduler bypass occurred within the controlled experiment.

For research computing organizations, the practical conclusion is straightforward. Existing schedulers, identity systems, filesystem permissions, and institutional policies should remain authoritative. Project instructions can guide the model toward site practices and reduce undesirable action attempts. Measured scheduler and performance evidence can ground resource recommendations. Consequential actions should be exposed through narrow, auditable, independently authorized capabilities.

The long-term objective is therefore not to place an autonomous AI scheduler on an HPC cluster. It is to provide an AI-enabled research computing facilitation layer that can translate researcher intent into efficient, understandable, evidence-based, and policy-compliant use of existing HPC services.

References

- [1] H. Ma, A. Brace, C. Siebenschuh, G. Pauloski, I. Foster, and A. Ramanathan, “Connecting Large Language Model Agent to High Performance Computing Resource,” arXiv:2502.12280, 2025. <https://arxiv.org/abs/2502.12280>
- [2] A. Kamatar, J. G. Pauloski, Y. Babuji, R. Chard, M. Sakarvadia, D. Babnigg, K. Chard, and I. Foster, “Empowering Scientific Workflows with Federated Agents,” in *2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 1403–1418, 2026, doi:10.1109/IPDPS65963.2026.00114.
- [3] T. D. Pham, H. Tummalapalli, F. H. Bhuiyan, A. Vázquez Mayagoitia, C. M. Simpson, R. Balin, V. Vishwanath, and M. Keçeli, “Multi-Agent Orchestration for High-Throughput Materials Screening on a Leadership-Class System,” arXiv:2604.07681, 2026. <https://arxiv.org/abs/2604.07681>
- [4] N. Shahin and I. Alsmadi, “HPC-LLM: Practical Domain Adaptation and Retrieval-Augmented Generation for HPC Support,” arXiv:2605.16347, 2026. <https://arxiv.org/abs/2605.16347>
- [5] S.-i. Hayashi, K. Morita, D. Mukunoki, T. Hoshino, and T. Katagiri, “VibeCodeHPC: An Agent-Based Iterative Prompting Auto-Tuner for HPC Code Generation Using LLMs,” arXiv:2510.00031, 2025. <https://arxiv.org/abs/2510.00031>
- [6] M. Zheng, F. Linghu, S. Li, M. C. Kandes, N. Gaffney, I. T. Foster, and Z. Zhang, “A Comparative Study on LLM-enabled HPC User Support,” in *Proceedings of the Practice and Experience in Advanced Research Computing 2026 (PEARC ’26)*, Article 9, pp. 1–8, 2026, doi:10.1145/3785462.3815796.
- [7] Model Context Protocol, “Model Context Protocol Specification, Revision 2026-07-28,” July 28, 2026. <https://modelcontextprotocol.io/specification/2026-07-28>
- [8] H. Pan, R. Chard, R. Mello, C. Grams, T. He, A. Brace, O. P. Skelly, W. Engler, H. Holbrook, S. Y. Oh, M. Gonthier, M. E. Papka, B. Blaiszik, K. Chard, and I. T. Foster, “Experi-

- ences with Model Context Protocol Servers for Science and High Performance Computing,” arXiv:2508.18489, 2025. <https://arxiv.org/abs/2508.18489>
- [9] C. Ehrett, A. Feltus, D. Dawson, Z. Gerstner, and T.-Y. Chu, “BioCodex: HPC-Native Agentic Genomics Workflows via MCP RunSpecs and Asynchronous Slurm Execution,” in *Proceedings of the Practice and Experience in Advanced Research Computing 2026 (PEARC ’26)*, Article 57, pp. 1–4, 2026, doi:10.1145/3785462.3815873.
 - [10] J. Li, “Trusted Credentials, Untrusted Behavior: Benchmarking LLM-Agent Security in High-Performance Computing,” arXiv:2607.18485, 2026. <https://arxiv.org/abs/2607.18485>
 - [11] R. Souza, A. Gueroudji, S. DeWitt, D. Rosendo, T. Ghosal, R. B. Ross, P. Balaprakash, and R. Ferreira da Silva, “PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows,” in *2025 IEEE International Conference on eScience (eScience)*, pp. 467–473, 2025, doi:10.1109/eScience65000.2025.00093.
 - [12] J. Weiner, “PSI – Pressure Stall Information,” Linux kernel documentation, Apr. 2018. <https://docs.kernel.org/accounting/psi.html>
 - [13] T. Heo, “Control Group v2,” Linux kernel documentation, Oct. 2015. <https://docs.kernel.org/admin-guide/cgroup-v2.html>
 - [14] SchedMD, “Control Group in Slurm,” Slurm Workload Manager documentation. <https://slurm.schedmd.com/cgroups.html>
 - [15] SchedMD, “Control Group v2 plugin,” Slurm Workload Manager documentation. https://slurm.schedmd.com/cgroup_v2.html
 - [16] S. L. Pendelberry, “stall-pressure: Improving HPC Job Performance through Resource Utilization and Stall Pressure Management,” GitHub repository, accessed Sept. 4, 2026. <https://github.com/sidpbury/stall-pressure>